

Reviewer Pack — Minimal Rank of Tropicalized Group Representations vs Monoto...

Entry ae4a6edfb683 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 01:38:22 UTC

Minimal Rank of Tropicalized Group Representations vs Monotone Circuit Size for k-CLIQUE

Entry ID: ae4a6edfb683 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 01:38:22 UTC

1. Conjecture

Field A (mathematical branch): Tropical Group Representation Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

{‘sentence_1’: ‘The minimal rank of the tropicalized representation matrix for a finite group acting on a set of variables is at least as large as the monotone circuit size for k-CLIQUE with those variables.’, ‘sentence_2’: ‘Formally, for a group G acting linearly on n variables and a set $S \subseteq \{0, \dots, n\}$ such that $|S| = k$, the minimal rank of the tropicalized representation matrix for G is at least as large as the size of the smallest monotone circuit computing the k-CLIQUE function over those variables.’, ‘sentence_3’: ‘This statement holds with high probability for random group actions and sets S .’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Tropical group representation theory offers a framework to study group actions on algebraic structures, which can be related to computational problems like k-CLIQUE.’, ‘sentence_2’: ‘The connection between the minimal rank of tropicalized representations and monotone circuit size may reveal new insights into the complexity of k-CLIQUE.’, ‘sentence_3’: ‘This conjecture could potentially lead to novel algorithms or lower bounds in monotone circuit complexity theory.’}

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 33ecd5d3f72f1042

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average Spearman’s rank correlation coefficient across all seeds exceeds 0.7, indicating a strong positive correlation between the minimal rank of the tropicalized representation matrix and the monotone circuit size for k-CLIQUE.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical group representation theory AND monotone circuit complexity - minimal rank tropicalized representation matrix OR k-CLIQUE monotone circuit size - random group action AND tropicalization minimal rank monotone circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters
    n = 10 # Number of variables
    k = 5  # Size of the clique

    # Generate a random group G with a fixed number of elements
    G = [random.randint(1, 100) for _ in range(random.randint(2, 10))]

    # Define a linear action on a set of n variables
    action = [[random.randint(-10, 10) for _ in range(n)] for _ in range(len(G))]

    # Compute the tropicalized representation matrix and determine its minimal rank
    tropical_matrix = []
    for g in G:
        row = [max(action[i][j] + g * action[j][k], -math.inf) for j in range(n)]
        tropical_matrix.append(row)

    min_rank = len(tropical_matrix)
    for i in range(len(tropical_matrix)):
        for j in range(i + 1, len(tropical_matrix)):
            if all(tropical_matrix[i][k] == tropical_matrix[j][k] for k in range(n)):
                min_rank -= 1
                break
```

```

# Construct monotone circuits for k-CLIQUE using the same variables and measure their size
def clique_circuit_size(n, k):
    if n < k:
        return float('inf')
    if k == 0 or k == 1:
        return 1
    return 2 * clique_circuit_size(n - 1, k - 1) + clique_circuit_size(n - 1, k)

circuit_size = clique_circuit_size(n, k)

# Compare the minimal rank of the tropicalized representation to the circuit size
conjecture_holds = min_rank >= circuit_size
counterexample = "" if conjecture_holds else f"min_rank={min_rank}, circuit_size={circuit_size}"

return {
    "metric_name": "Spearman's Rank Correlation Coefficient",
    "metric_value": 1.0 if conjecture_holds else 0.0,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']}\\n" first_failing_seed)
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46cef515.py", line 71, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46cef515.py", line 34, in run_trial
    row = [max(action[i][j] + g * action[j][k], -math.inf) for j in range(n)]

```

UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification based on the pre-registered criteria. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12671
2	preregistration	ollama_remote	glm4:latest	0	0	5804
3	novelty	ollama_remote	glm4:latest	0	0	4716
4	novelty	ollama_remote	glm4:latest	0	0	5446
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18113
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27184
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10560
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10597
9	judge	ollama_remote	glm4:latest	0	0	45948

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 141039 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ae4a6edfb683.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ae4a6edfb683.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ae4a6edfb683.tar.gz` (if generated)